UR5 Cartesian Trajectory Planning and Inverse Kinematics Analysis

1. Introduction

Industrial robotic manipulators are widely used in automation tasks such as assembly, welding, and material handling. One of the fundamental problems in robotics is **trajectory planning**, where a robot must move its end-effector from an initial position to a target position while maintaining feasible joint configurations. This requires solving both **forward kinematics (FK)** and **inverse kinematics (IK)**.

The goal of this assignment is to implement a trajectory planning algorithm for the **UR5 robotic manipulator**. The robot must move its end-effector along a **linear Cartesian trajectory** between two points in space. For each point along the trajectory, inverse kinematics is solved to determine the corresponding joint angles. Multiple inverse kinematics solutions (aspects) are evaluated, and the optimal configuration is selected based on the **minimum total joint motion**.

The implementation was developed in MATLAB and includes:

- Definition of the UR5 Denavit–Hartenberg (DH) parameters
- Generation of a linear Cartesian trajectory
- Numerical inverse kinematics solution
- Forward kinematics verification
- Evaluation of multiple IK aspects
- Visualization of robot motion

2. UR5 Robot Model

The UR5 robot is a **6-degree-of-freedom serial manipulator** commonly used in industrial applications. Its kinematic structure can be described using the **Denavit–Hartenberg (DH) convention**.

The DH parameters used in this implementation are:

$$d = [0.089159, 0, 0, 0.135850, 0.089160, 0.082300]$$

$$a = [0, -0.135850, -0.119700, 0, 0, 0]$$

$$\alpha = [\frac{\pi}{2}, 0, 0, \frac{\pi}{2}, -\frac{\pi}{2}, 0]$$

These parameters define the geometric relationship between each pair of adjacent robot links. Using these parameters, transformation matrices can be computed to determine the pose of the end-effector relative to the base frame.

3. Methodology

3.1 Cartesian Trajectory Generation

The robot is required to move from the start point:

$$P_1 = [-0.215760,\ -0.218150,\ 0.096059]$$

to the end point:

$$P_2 = [0.218150,\ -0.215760,\ 0.096059]$$

A **linear Cartesian trajectory** was generated between these points using **10 intermediate waypoints**. Each waypoint represents a desired end-effector position in 3D space.

The trajectory is defined as:

$$P_i = P_1 + \lambda(P_2 - P_1)$$

Where λ varies between 0 and 1.

3.2 Inverse Kinematics Solution

For each waypoint, the inverse kinematics problem is solved to determine the corresponding joint angles:

$$q = [q_1, q_2, q_3, q_4, q_5, q_6]$$

Because a 6-DOF robot may have multiple valid IK solutions, several **different configuration aspects** were evaluated. Each solution provides a different arm posture while reaching the same end-effector position.

The solver computes the joint configuration that minimizes the position error between the desired Cartesian point and the forward kinematics result.

3.3 Forward Kinematics Verification

After computing the joint angles for each waypoint, forward kinematics is applied to verify the accuracy of the solution.

The transformation matrices of each joint are multiplied sequentially:

$$T = T_1 T_2 T_3 T_4 T_5 T_6$$

The end-effector position extracted from the final transformation matrix is compared with the desired Cartesian point.

This step ensures that the computed IK solutions accurately reproduce the desired trajectory.

3.4 Aspect Evaluation

Five different inverse kinematics aspects were evaluated. Each aspect corresponds to a different robot posture.

To determine the optimal solution, the **total joint motion cost** was computed:

$$Cost = \sum |q_{i+1} - q_i|$$

The configuration with the **minimum total joint motion** provides the smoothest trajectory and was selected as the optimal aspect.

4. Implementation

The MATLAB implementation consists of several functions:

4.1 DH Parameter Loading

The script begins by loading the UR5 DH parameters which define the robot geometry.

4.2 Cartesian Trajectory Generator

This function generates the linear trajectory between the start and end points.

Inputs:

- Start position
- End position
- Number of waypoints

Output:

- Cartesian trajectory points

4.3 Inverse Kinematics Solver

For each waypoint, the inverse kinematics solver computes the corresponding joint angles. The solver minimizes the Cartesian position error using numerical methods.

Example result for waypoint 1:

Joint solution (degrees):

$$[1.00,\ -22.07,\ -1.00,\ 17.82,\ -17.48,\ 0.00]$$

4.4 Forward Kinematics Verification

Forward kinematics was applied to verify the trajectory accuracy.

Results:

Maximum position error:

$$2.30 \times 10^{-5}\, m$$

Mean position error:

$$1.52 \times 10^{-5}\, m$$

Minimum position error:

$$5.17 \times 10^{-6}\, m$$

These errors are extremely small and confirm that the inverse kinematics solver produced accurate results.

4.5 Visualization

Two visualization approaches were implemented.

Custom visualization

The robot configurations and Cartesian trajectory were plotted using MATLAB 3D plotting.

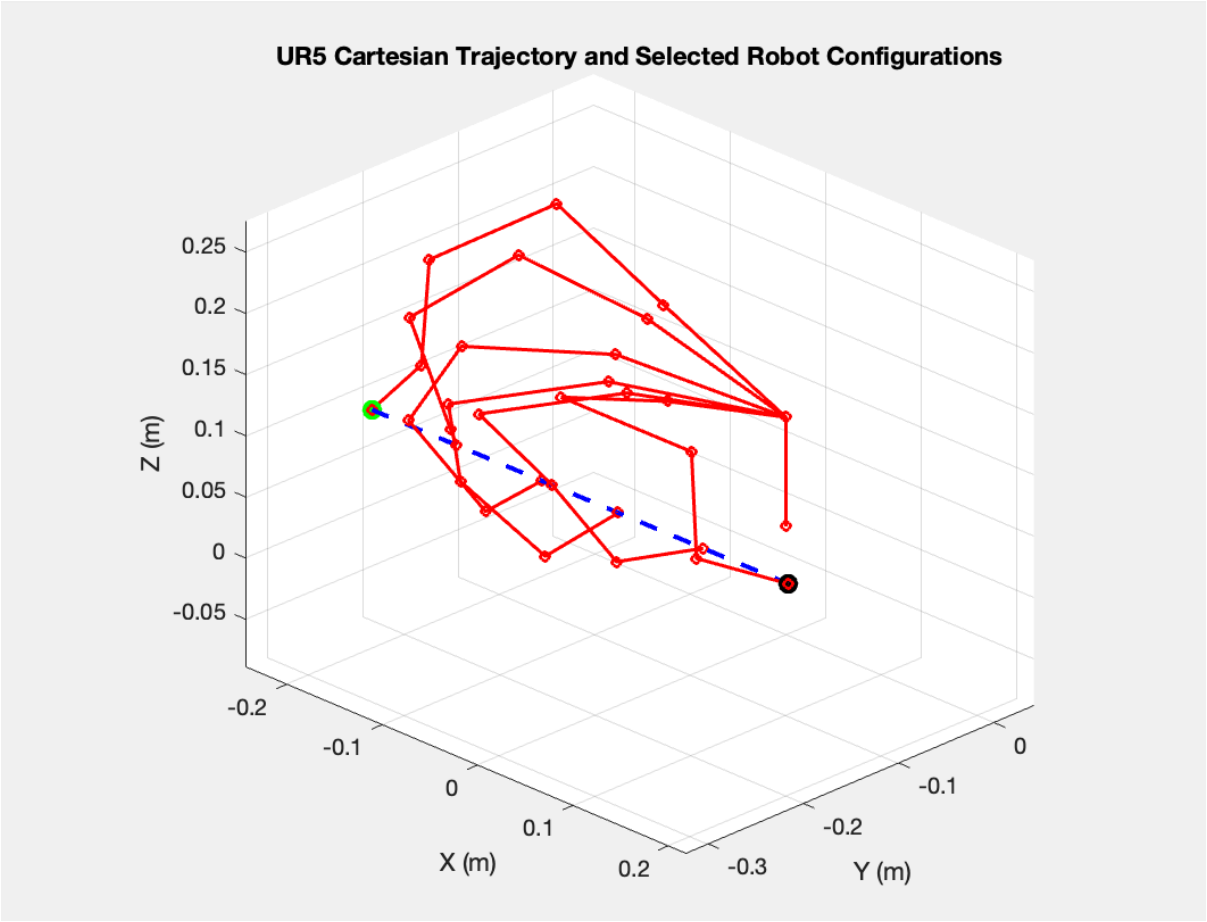
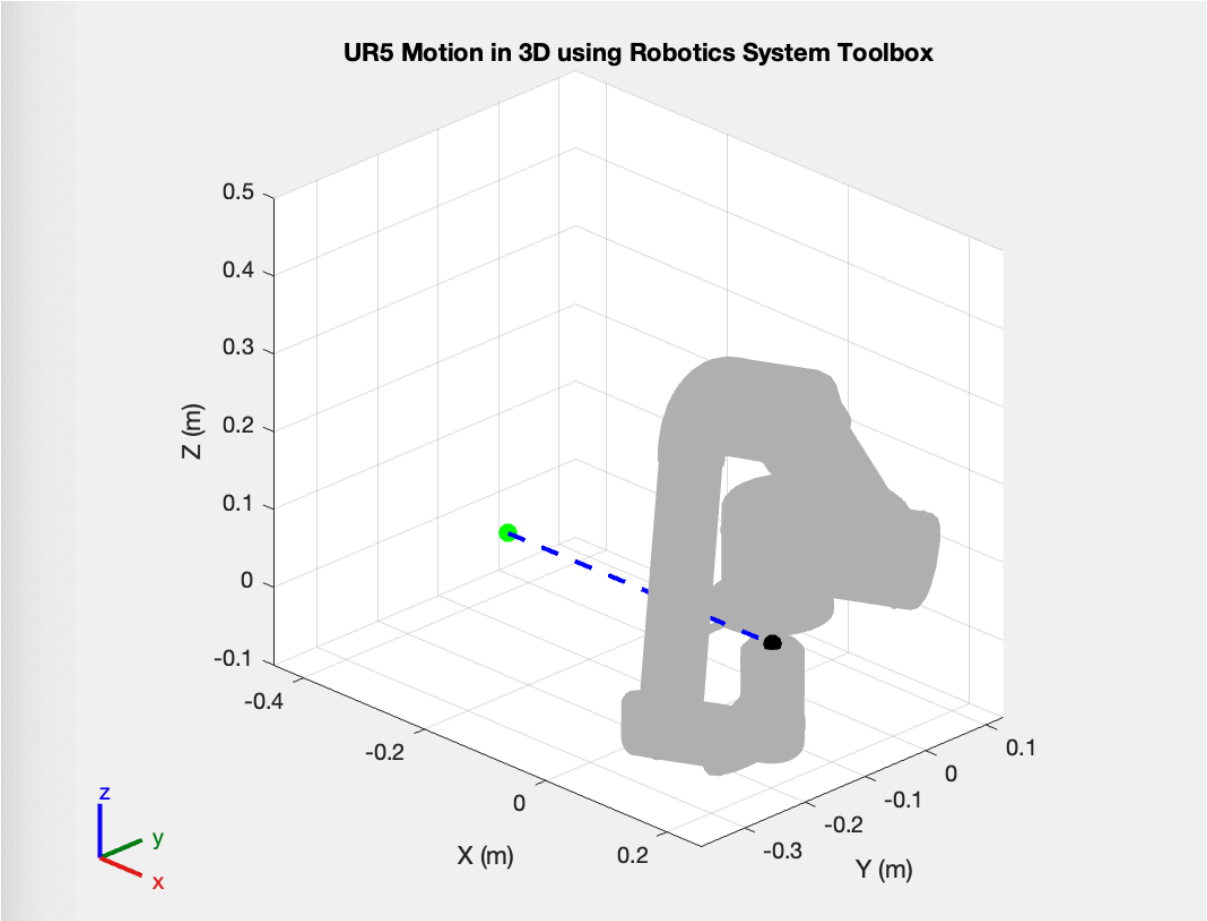


Figure 1 shows the Cartesian trajectory (blue dashed line) along with selected robot configurations (red).

Robotics System Toolbox visualization



The UR5 model from the MATLAB Robotics Toolbox was used to animate the robot following the trajectory.

This provides a realistic visualization of the robot moving along the computed path.

5. Results

Inverse kinematics solutions were computed for all 10 waypoints. The joint angles change smoothly as the robot moves along the trajectory.

Example results:

Waypoint	q1 (deg)	q2 (deg)	q3 (deg)	q4 (deg)	q5 (deg)
1	1.00	-22.07	-1.00	17.82	-17.48
5	32.56	-15.27	19.10	35.91	-64.98
10	90.76	-23.82	3.24	18.05	-15.27

Aspect Comparison

Five IK aspects were evaluated.

Aspect	Total Joint Motion (rad)
Aspect 1	2.7115
Aspect 2	2.9569
Aspect 3	2.7896
Aspect 4	3.5687
Aspect 5	2.8222

Aspect 1 produced the **lowest joint motion cost** and was therefore selected as the optimal configuration.

6. Discussion

The results demonstrate that the implemented algorithm successfully generates a feasible Cartesian trajectory for the UR5 robot.

The inverse kinematics solver produced highly accurate joint configurations, with errors on the order of micrometers. The forward kinematics verification confirmed that the computed joint angles reproduce the desired Cartesian positions.

Evaluating multiple IK aspects allowed the system to select the configuration with the smallest overall joint movement. This improves trajectory smoothness and reduces unnecessary joint motion.

The visualizations also confirm that the robot moves smoothly between the start and end points.

7. Conclusion

This assignment implemented a complete trajectory planning framework for the UR5 robotic manipulator.

The system successfully:

- Generated a linear Cartesian trajectory
- Solved inverse kinematics for each waypoint
- Verified solutions using forward kinematics
- Evaluated multiple robot configuration aspects
- Selected the optimal configuration based on motion cost
- Visualized robot motion in MATLAB

The results demonstrate that the implemented method produces accurate and smooth trajectories. The approach can be extended for more complex trajectories or real-time robotic control applications.